

# Programação Orientada a Objetos

MySQL, JDBC e Conexão

[Baixe aqui os exemplos da aula](#)



**Prof. Everaldo Graciliano Souza Ribeiro**

Especialista em Desenvolvimento de Sistemas Para a Internet

**Universidade Federal de Viçosa**

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

# Da Memória ao Disco

Desde a Aula 01, o GestorOS guarda clientes, técnicos e ordens de serviço apenas na memória do processo Java: ao encerrar o programa, cada objeto desaparece. Um sistema de gestão de verdade precisa sobreviver ao fechamento da aplicação — os dados cadastrados hoje devem continuar disponíveis na próxima execução. Essa exigência motiva a introdução do MySQL como representação persistente do mesmo domínio já modelado nas classes Java.



**Prof. Everaldo Graciliano Souza Ribeiro**

Especialista em Desenvolvimento de Sistemas Para a Internet

**Universidade Federal de Viçosa**

 [everaldoribeiro01@gmail.com](mailto:everaldoribeiro01@gmail.com)

 <https://www.linkedin.com/in/everaldoribeiroads>

# JDBC: Um Contrato Para Qualquer Banco

JDBC é a API padrão do próprio Java para conversar com bancos relacionais: define interfaces como `Connection`, `Statement` e `ResultSet` sem amarrar o código a um SGBD específico. O mesmo código que abre uma conexão, executa um comando e lê um resultado funciona contra MySQL, PostgreSQL ou outro banco — desde que exista, por baixo da API, um driver que traduza essas chamadas para o protocolo daquele banco.



**Prof. Everaldo Graciliano Souza Ribeiro**

Especialista em Desenvolvimento de Sistemas Para a Internet

**Universidade Federal de Viçosa**

 [everaldoribeiro01@gmail.com](mailto:everaldoribeiro01@gmail.com)

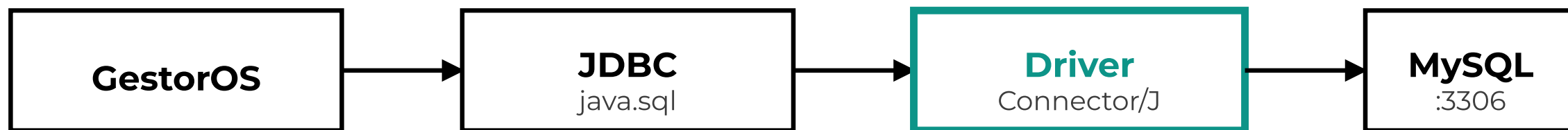
 <https://www.linkedin.com/in/everaldoribeiroads>

# O Driver e a URL de Conexão

O driver JDBC é a peça concreta, específica de cada SGBD, que implementa as interfaces genéricas da API: sem ele disponível, o DriverManager não sabe a quem entregar um pedido de conexão. A URL `jdbc:mysql://localhost:3306/gestoros` identifica, nessa ordem, o protocolo, o host onde o MySQL escuta, a porta padrão 3306 e o nome do banco — os dados que substituem, no mundo relacional, o caminho de um arquivo.



# As Camadas Entre o Código e o Banco



O GestorOS só conhece a API. Trocar de banco significa trocar apenas o **driver**, nunca o código que chama JDBC.



**Prof. Everaldo Graciliano Souza Ribeiro**

Especialista em Desenvolvimento de Sistemas Para a Internet

**Universidade Federal de Viçosa**

everaldoribeiro01@gmail.com

<https://www.linkedin.com/in/everaldoribeiroads>

# Connection É um Recurso, Não um Dado

`DriverManager.getConnection(url, usuário, senha)` devolve um `Connection` que representa um canal aberto e ativo com o servidor MySQL — não um valor qualquer, mas um recurso caro e limitado, que precisa ser liberado explicitamente. Por isso `Connection` implementa `AutoCloseable` e entra num `try-with-resources`, exatamente como um arquivo: o Java garante o fechamento ao final do bloco, mesmo que uma exceção interrompa o caminho normal.



**Prof. Everaldo Graciliano Souza Ribeiro**

Especialista em Desenvolvimento de Sistemas Para a Internet

**Universidade Federal de Viçosa**

 [everaldoribeiro01@gmail.com](mailto:everaldoribeiro01@gmail.com)

 <https://www.linkedin.com/in/everaldoribeiroads>

# SQLException: uma Falha do Mundo Externo

Toda operação que atravessa a rede até um SGBD pode falhar por motivos fora do controle do programa: o servidor pode estar fora do ar, a senha pode estar errada, a rede pode cair no meio da chamada. Por isso os métodos de JDBC declaram SQLException, uma exceção verificada — o compilador obriga cada chamador a decidir, no código, o que fazer quando o mundo externo não cooperar. A Aula 07 formaliza esse tratamento.

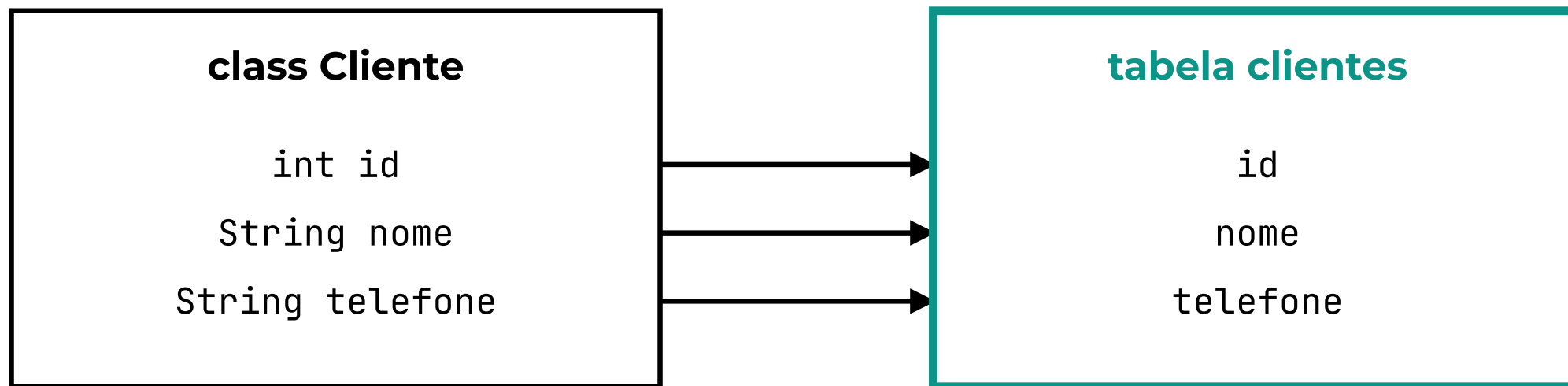


# Do Objeto à Tabela

Cada atributo encapsulado da classe Cliente — id, nome, telefone — corresponde a uma coluna da tabela clientes que o schema desta aula cria no MySQL: mesma informação, duas representações. A classe existe em tempo de execução, dentro da memória do programa; a tabela existe em disco, entre uma execução e outra. Fazer as duas convergirem é o trabalho que um DAO relacional assume na Aula 07.



# Cliente (Classe) → clientes (Tabela)



Mesmos três campos, duas representações. Nenhum Dao ainda faz essa travessia — é o que a Aula 07 constrói.



**Prof. Everaldo Graciliano Souza Ribeiro**

Especialista em Desenvolvimento de Sistemas Para a Internet

**Universidade Federal de Viçosa**

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

# Uma Classe Só Para Abrir a Conexão

Repetir a URL, o usuário e a senha em cada trecho do código que precisar do banco criaria duplicação e credenciais espalhadas, além de transformar qualquer mudança de configuração numa caça por todo o projeto. `ConexaoMySQL` concentra esse conhecimento num único ponto — a mesma ideia de `DaoBase` escondendo, sozinha, como o identificador de um T é lido e definido, aplicada agora à abertura da conexão.



# O DaoBase Ainda Não Muda

A conexão nasce nesta aula, mas o contrato `Dao<T>` e sua implementação `DaoBase` continuam exatamente como a Aula 05 os deixou: uma lista em memória, sem nenhuma linha de SQL. Trocar essa lista por uma implementação que grava e lê da tabela clientes é trabalho da Aula 07, junto do tratamento de exceções que uma operação de banco de verdade exige — hoje o alvo é só provar que o canal com o MySQL abre e fecha corretamente.



# banco/01\_schema.sql

-- Cada coluna espelha um atributo encapsulado de Cliente.

```
CREATE DATABASE IF NOT EXISTS gestoros;
```

```
USE gestoros;
```

```
CREATE TABLE IF NOT EXISTS clientes (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    nome          VARCHAR(120) NOT NULL,  
    telefone      VARCHAR(20)  NOT NULL  
);
```



**Prof. Everaldo Graciliano Souza Ribeiro**

Especialista em Desenvolvimento de Sistemas Para a Internet

**Universidade Federal de Viçosa**

 everaldoribeiro01@gmail.com

 <https://www.linkedin.com/in/everaldoribeiroads>

# util/ConexaoMySQL.java

```
public final class ConexaoMySQL {  
  
    private static final String URL =  
        "jdbc:mysql://localhost:3306/gestoros";  
    private static final String USUARIO = "root";  
    private static final String SENHA = "";  
  
    private ConexaoMySQL() {  
        // So metodos estaticos: nenhuma instancia faz sentido aqui.  
    }  
  
    public static Connection obter() throws SQLException {  
        return DriverManager.getConnection(URL, USUARIO, SENHA);  
    }  
}
```



# util/TesteConexao.java

```
try (Connection conexao = ConexaoMySQL.obter();
    Statement comando = conexao.createStatement();
    ResultSet resultado = comando.executeQuery("SELECT 1")) {

    if (resultado.next()) {
        System.out.println(
            "Conexao com o MySQL funcionando. Retorno: "
            + resultado.getInt(1));
    }
} catch (SQLException e) {
    System.out.println("Falha ao conectar: " + e.getMessage());
}
```



# Atividades

## Exercício 1

Por que Connection precisa estar dentro de um try-with-resources, e não apenas ser fechada "quando o programa lembrar"?

## Exercício 2

O que a URL jdbc:mysql://localhost:3306/gestoros diz ao DriverManager, e o que aconteceria se a porta estivesse errada?

## Exercício 3

Por que o DaoBase criado na Aula 05 continua guardando os registros em memória, mesmo já existindo uma conexão funcionando com o MySQL?



# Desafio extra

## Opcional

Quem quiser ir além do combinado nesta semana:

1. Escrever banco/03\_tabela\_tecnicos.sql, criando uma tabela tecnicos com as mesmas colunas de clientes mais uma coluna especialidade.
2. Alterar TesteConexao para imprimir a versão do MySQL, usando `SELECT VERSION()` no lugar de `SELECT 1`.

Nada disso é cobrado na entrega. O projeto segue completo sem essas adições.

